

02-614 Week 2

String Algorithms

Aidan Jan

September 8, 2026

Seminumerical String Algorithms

Rabin-Karp Fingerprinting

This is an exact matching algorithm exploiting the fact that characters are stored in a computer as binary. This makes it a *seminumerical* string algorithm.

Recall the exact matching problem: text T , pattern P . Find all occurrences of P in T .

For Rabin-Karp Fingerprinting, let

- $\Sigma = \{“0”, “1”, \dots, “9”\}$. (Note these are strings, not integers.)
- $T = “4278931”$ (Note this is a string, not a integer.)
- $P = “78”$

Now, since P has length 2, consider every substring of T of length 2. $t_i = (“42”, “27”, \dots, “31”)$. If we convert all substring in T to integers (so we have an integer array), and we convert P to an integer, p , then doing the matching is trivial. This problem essentially becomes finding when an integer appears in a string. The issue is that generating that integer array is not trivial.

Converting P to p

Let $m = |P|$. Then, (assume 1-indexed),

$$p = P[m] + 10(P[m-1] + (10P[m-2] + \dots + P[1]))$$

This formula is known as *Horner's rule*.

- Note that none of the numbers in this formula is larger than the size of our alphabet.
- Each character in the string using this formula is $O(1)$ work, since it is a single multiplication in the recursive formula.
- Therefore, $P \rightarrow p$ runs in $O(|P|)$ time.

Converting T to t

Let

$$t_i = 10 \times (t_{i-1} - 10^{m-1}T[i-1]) \times T[s+m-1]$$

This formula is similar to Horner's rule; it essentially does:

- Calculate the first integer substring, t_1

- Subtract the leading digit of the first integer, multiply by 10 (or size of alphabet), and add the next integer to generate t_2
- Repeat until the end of the string to calculate all the t_i values.

This makes the array of integers in $O(|T|)$ time, since each t_i value occurs in constant time. Once again, it is a single multiplication per character.

- Note that this would be even faster in binary, where the alphabet size is 2. In this case, multiplications become bit shifts.

Issues With This Method

The primary issue is that we did not state when computers are good at multiplication. If T is a billion base-pair genome and P is a thousand base-pair read, then the computer will struggle to multiply anything.

Problem: t_i and p could be huge numbers. Instead, we can do the following:

- Define $t'_i \equiv t_i \pmod{8}$
- Define $p' \equiv p \pmod{8}$
- P occurs at point i in T implies $t'_i = p'$. Unfortunately, this implication only goes one direction, and we cannot assume the opposite.
- $t'_i = p'$ does **NOT** imply that P occurs at i . We will now have false positives.

Dealing with False Positives

Let q be a prime number less than or equal to M . Let M be a number that we pick such that any q less than M is small enough to fit into our computer's registers. Now, choose a q randomly.

- We deal with the false positive issue by adding some randomness. So essentially, false positives would only occur if we get really unlucky.

Theorem: if M is big enough, then

$$\Pr_q[t'_i = p'] \leq 0.2 \text{ when } T[i \dots] \neq P$$

This is essentially, “given our choice of q , what is the probability we get a false positive?” t and p are given by the input, and have nothing to do with the probability.

Proof:

1. Think of P and T as binary strings. Define $S = T[i \dots]$, a substring of length P .
2. $p \leq 2^N$ and $t_i \leq 2^N$, where $N = |P|$ (We have two symbols for the length of P now.) Basically the numbers represented in less than N bits. These are our un-*mod*'ed numbers.
3. Define a false positive (FP) when $t'_i = p'$ if and only if $(t_i - p) \equiv 0 \pmod{q}$. In other words, q divides $|t_i - p|$. Let $|t_i - p| = D$. Now, we rephrase the question: given D , how many prime q 's can divide it evenly?
4. $D \leq 2^N$. This must be true because $D = |t_i - p|$, which both numbers must be smaller than 2^N . Now, we write $D = q_1 q_2 q_3 \dots q_k$, where $q_i \forall i < k$ are primes. This is a prime factorization of D . k is the number of prime factors. We know that $k \leq N$ since $D \leq 2^N$, and the prime factorization of D is maximal when every factor is a 2. This places an upper bound on the number of numbers that divide D .
5. The probability we picked a “bad” q is the number of (bad choices / possible choices). This fraction must be less than $\frac{N}{\text{number of primes less than } M} \leq M$.

6. Pick M so there are $5N$ primes less than or equal to M .

$$\Rightarrow \frac{\# \text{ bad}}{\# \text{ choices}} \leq \frac{N}{5N} = 0.2$$

7. QED.

The theorem allows us to pick an M , so we do so. We could pick the theorem to be any arbitrary number by changing our decision of M . However, we want M to be big for the theorem, but also we cannot make it too big. We now need a practical way to pick M .

Fact: If we want $n \geq 4$ primes between 1 and M , it suffices to choose $M \geq 2n \log_2 n$. We will not be proving this.

- In our application, we need $5N$ primes less than M . Therefore, we need $M \geq 2 \cdot (5N) \log_2(5N) = 10N \log_2(5N)$. This is an upper bound on the size of M .
- This implies that we need $O(\log M)$ bits to represent our q . Since $q \leq M$, all our numbers are bounded by M bits.
- Plugging in, we get $O(\log(10N \log(5N))) = O((\log 10N) + (\log \log 5N)) = O(\log N)$. N is the length of our pattern!
- This is definitely fast enough to be feasible for our use case of trying to fit all the computation in a register, because just to index T (e.g., write down $T[i]$), at least $\log N$ bits are required.

The last thing we want to address is the size of the constant, 0.2. It is a *constant*, but it is also large enough that 20% false positives is kind of bad. There are two ways to decrease the constant: increase M , or repeat picking a prime R times instead of just once.

- Increasing M increases the constant in front of the space complexity, which might not be possible if we are trying to fit registers.
- However, we can roll more times at a constant time cost instead.

The Shift-And Algorithm

Another algorithm for exact matching. For this algorithm, we create a matrix $M = \{0, 1\}^{m \times n}$, where m is the length of P and n is the length of T .

- The rule for filling out the matrix is $M[i, j] = 1$ if and only if $P[i] == T[j]$ and $P[1, i - 1] == T[\text{ending at } j - 1]$.
- The answer is the bottom row of the matrix.
- This is reminiscent of dynamic programming; the box at $M[i, j]$ depends on $M[i - 1, j - 1]$ and whether the characters in the string match.
- Unfortunately, a naive method of filling in this matrix is slow. ($O(|T|^2)$). We need a faster way.

Filling in the Matrix Fast

If instead of filling out the cells one at a time, we filled it using vectorized operations instead, then we can make this run in $O(|T|)$.

- Notice that each column in the matrix looks at the same ending character in T .
- Since every cell is based on its up-left diagonal, we can do a Shift-And. Essentially, take the column to the left, shift the entire column down and right by one, and AND the entire binary vector.

- We can now write the fill-out operation as

$$M[\cdot, j] = U(T[j]) \ \& \ (M[i, j-1] \gg 1)$$

- where $U(x)$ is a $|P|$ -long bit vector, such that $U(x)[i] == 1$ if and only if $P[i] == x$.
- Example: if $P = AACCTGA$, then $U("A") = 1100001$.

Time Complexity

Even when we vectorize the algorithm, does bitwise-AND run in constant time?

- The answer depends on how big P is. If the entire binary vector is less than length 32 (e.g., P is 32 characters long), then the entire vector may fit into a single register.
- This allows the algorithm to be hardware-optimized, and run in $O(n)$ time if P is short enough.
- In practice, this algorithm runs as efficiently as (or even better than) the others of the conditions are met.

Approximate Matching

ℓ -mismatch problem: Given T , P , and L , find all places in T where $T == P$, up to L disagreements.

- L is the budget of how many mismatches we get.
- ℓ is how many mismatches we have.

Variation of Shift-And for Approximate Matching

Similar to the normal Shift-And, we also create a matrix, but in 3-dimensions. Every ℓ receives its own matrix, where

- $M^\ell[i, j] := 1$ if $P[1 \dots i] = T[\text{ending at } j]$ using $\leq \ell$ mismatches.
- This time, our answer would be the bottom row of the L -th matrix. $M^L[|P|, \cdot]$

We have a similar recursion algorithm for dynamic programming:

$$M^\ell[j] = M^{\ell-1}[j] \ \text{OR} \ (M^\ell[j-1] \gg 1 \ \text{AND} \ U(T[j])) \ \text{OR} \ (M^{\ell-1}[j-1] \gg 1)$$

- There are three cases:
- The middle one is what we did before, just on one matrix of the ℓ matrices.
- The left one is the case where we use up a mismatch.
- The right one is the case where we add to our mismatch “budget” and use it on the next character.

Searching for Multiple Patterns

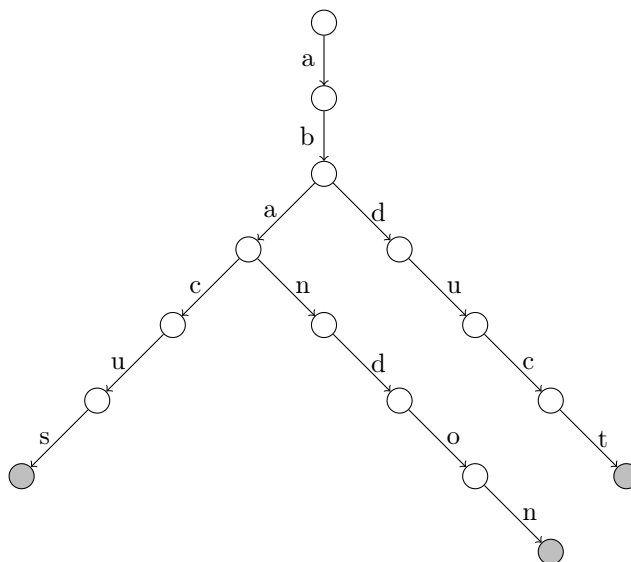
The problem statement: find every occurrence in T : $P_1, P_2, \dots, P_k$.

- If we ran KMP individually, (each string takes $O(|T| + |P|)$), then the total would be $O(|T| + |P_1| + |T| + |P_2| + \dots + |T| + |P_k|) = O(K|T| + \sum_i |P_i|)$
- The $\sum_i |P_i|$ part isn’t a problem, but the $K|T|$ is. We want to reduce the section of scanning over the input multiple times.
- However, there is a runtime caveat though, which is with multiple strings, the number of occurrences we print out at the end scales quadratically with the length of T . This is unavoidable since patterns can overlap.

Aho-Corasick Algorithm

We will define a keyword tree. Start with $K(P_1, P_2, \dots, P_k)$.

- For an example, let $P = \{\text{"abandon"}, \text{"abduct"}, \text{"abacus"}\}$.



- Let $L(v)$ be the string spelled out from the root to v .
- Let $\ell_p(v)$ be the longest proper suffix of $L(v)$ that is a prefix of *some* pattern
- Let $f(v)$ be the node representing $\ell_p(v)$
 - $f(v)$ is essentially a “failure node” or “failure path” that points backwards up the keyword tree. Usage of this will be seen later.

Using the Keyword Tree

Starting at the root node, use T as instructions as which branch of the tree to move down, similar to KMP.

- If we reach the bottom node, then it is an accept state for one of the strings in P .
- If we ever reach a point where we cannot take a path, then we take the “failure path” to jump backwards up the tree to an earlier node.
 - Note that this is very similar to the **memo** array from KMP. In fact, if our keyword tree only had one pattern to match, it would be exactly the same.
- Now, we just have to compute f .

But First, a Bug

What happens if $P_i \subseteq P_j$? That is, what if one pattern P_i is a substring of another pattern P_j . In this case, we could follow the path for P_j and completely miss P_i .

- To solve this problem, all we need to do is follow the failure function. Or rather check it. At any given point, if a pattern ends at the current character in T we are traversing, the pattern would be a suffix. The failure links have the purpose of matching suffixes to prefixes.
- However, there is a slight problem: The failure-links point to the *longest suffix* that matches the prefix. What if our pattern is not the longest?

- In this case, we can follow the failure link again, and continue doing this until we reach the root.

Now this introduces a new issue: if we have to follow all the failure links backwards to the root on each iteration, traversing them would take linear time..., for a total of quadratic time when iterating over the entire string.

- This is slow!

We can solve this problem by pre-processing the tree. In addition to the failure links (F-links), we introduce output links (O-links), which act like F-links but skip all nodes that are not leaves. In other words, they point from every node to all of the pattern matches. Note that we can derive the O-links from the F-links list.

- Now, we don't have to traverse the F-links on every iteration, we traverse the O-links instead.
- This might still take quadratic time in the worst case, but it is okay, because every node visited in the O-link represents an index that needs to be output.
- Our output, as established at the start, is un-improvably quadratic. Therefore, traversing O-links do not take extra time.

Runtime

This algorithm runs in $O(|T| + \sum_i |P_i| + z)$, where $|T|$ is the length of the string, $\sum_i |P_i|$ is the sum of the lengths of all the pattern strings, and $|z|$ is the size of our output.

- The z -term is quadratic by nature.
- This algorithm reads every character in our string T once. Building the keyword tree is in the $\sum_i |P_i|$ time, and traversing the tree is included in the $|z|$ term.

Computing the Failure Links, f

Suppose we can compute the failure links in levels $1, \dots, f(v)$, where v is a node somewhere in the tree, u is the child of v , and $f(x)$ refers to the level of x . Essentially, if we had F-links of all the nodes up to level $f(v)$, can we derive the F-links for node u in constant time?

- It turns out we can, and quite easily. Consider the string segments u and v in T , where u is one character longer than v . By definition, the last $f(v)$ characters of v match the first $f(v)$ characters of u .
- When we add a letter to the end of v to form u , x (the last character of u), we want to know the longest pattern in v that is extendable by x .
- Notice that this is the purpose of the failure link of v ! By definition, the failure link of v would give a prefix of v , so we can follow the failure links backwards until finding a pattern extendable by x .

Algorithm: To compute $f(u)$, follow $f^*(\text{parent}(u))$ until you find a node extendable by label of edge (v, u) .

Now, the inductive hypothesis is complete; we only need the base case. The base case happens to be the first character off of the root, in which case, the F-link of that node would point to the root.

What about the runtime?

- We might have to follow multiple F-links from v in order to find a node extendable by x .

Consider path Q spelling pattern P_i . We want to prove the total time to compute $f(u)$ for all u on Q is $O(|P_i|)$.

- First, note that $|Q| = |P_i|$.

- Define $\ell_p(u) = L(f(u))$. $\ell_p(u)$ is the string represented by $f(u)$. $L(f(u))$ is the label of $f(u)$. What we care about now is the length of the string, $|\ell_p(u)|$. Since u is parent of v , the length to v is at most 1 larger. That is, $\Delta|\ell_p| \leq 1$. For simplicity, we will measure length in ℓ_p (and $\Delta|\ell_p|$) in dollars.
 - Going down the path gains \$1. Following a F-link loses \$1. Suppose we traverse the entire path normally. If there are q nodes in path Q , we gain at most q dollars. However, when we follow F-links backwards, we can never go past the root of the tree. This implies the total amount of dollars at the end of a given path is \$0.
 - This in turn implies that the amortized amount spent on following all F-links equals the amount gained following the path Q . Therefore, the total time to compute all F-links must be within $O(\sum_i |P_i|)$.
 - In implementation, we don't compute the patterns one at a time. Rather, we compute all the patterns at the same time, following BFS order traversing the tree. However, since all paths are amortized to their length, the entire tree would run in linear time.